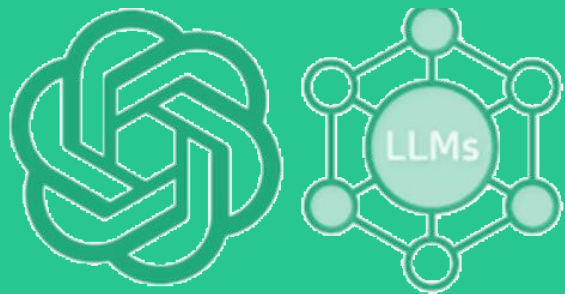


Langchain y LLM: Desarrolla Aplicaciones de IA en Python con Langchain



LangChain





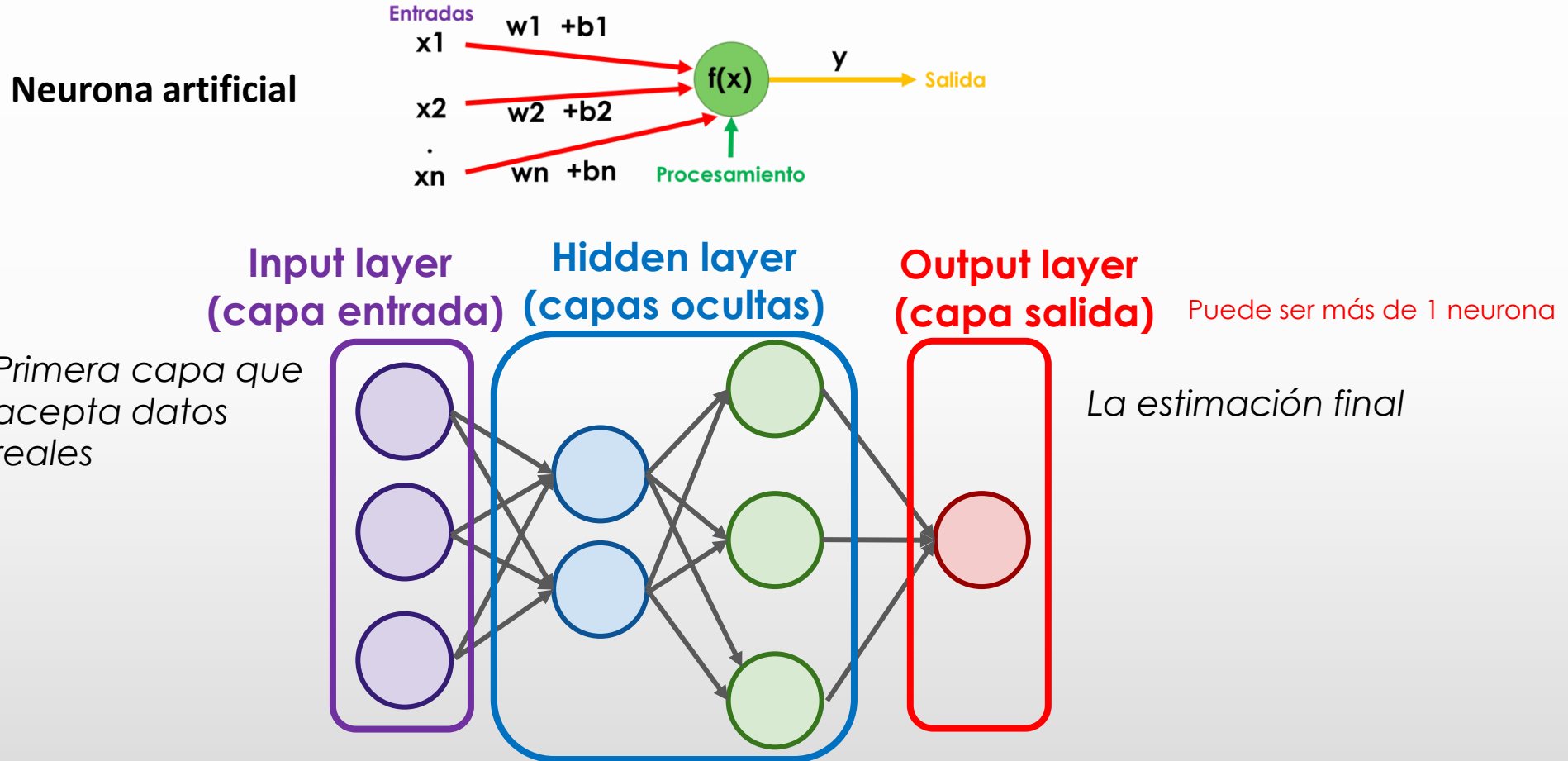
UNIDAD: INTRODUCCIÓN A LANGCHAIN Y LLMs

Lo que aprenderemos en este bloque....

- ✓ ¿Qué es un Large Language Model (LLM)?
- ✓ ¿Qué características y fases tiene un LLM?
- ✓ ¿Qué LLMs existen actualmente y cuáles son sus aplicaciones?
- ✓ ¿Qué es ChatGPT y cómo registrarnos?
- ✓ ¿Cuáles son los primeros pasos en ChatGPT?

¿Qué es un Large Language Model (LLM)?

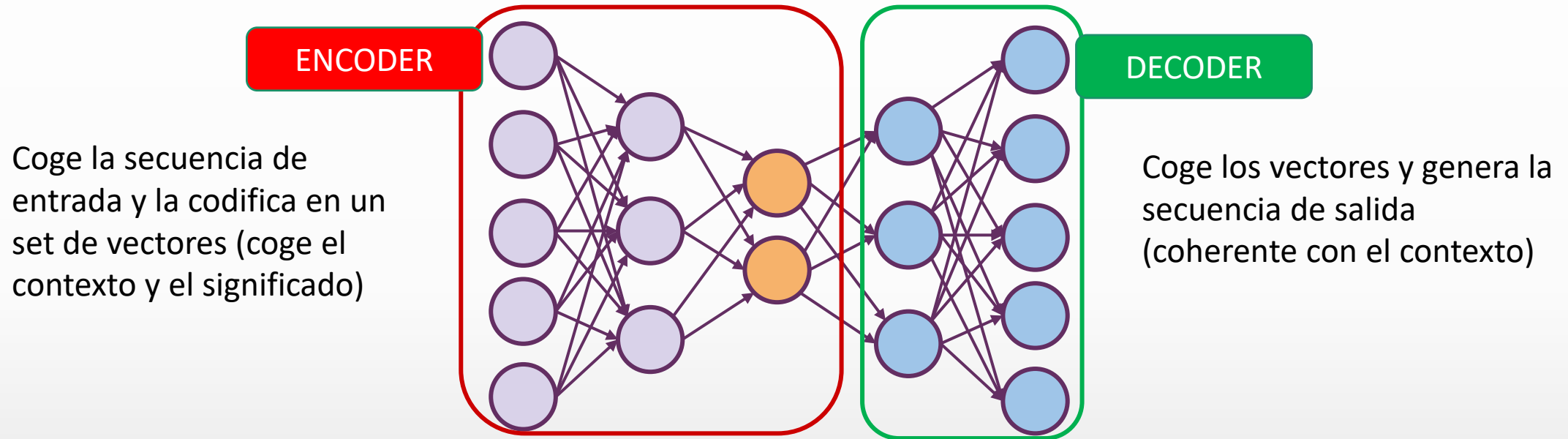
- Una red neuronal es un método de la inteligencia artificial que enseña a las computadoras a procesar datos de una manera que **está inspirada en la forma en que lo hace el cerebro humano y sus neuronas**.



- En un **LLM** se entrena la red neuronal con una cantidad **ingente** de datos recopilados de internet, patrones de lenguaje, estructuras y detalles del lenguaje.

¿Qué es un Large Language Model (LLM)?

- En un LLM aparece el concepto del modelo “**transformer**” que aplica a secuencias de datos (como los son las cadenas de texto).



- “**Mecanismo de atención**”: Permite que el modelo **se centre en diferentes partes de la entrada** para generar cada palabra de la secuencia de salida. Cada palabra de la entrada tiene un **peso** que representa su importancia en el contexto completo (capa multi-head).
- Efecto Alucinación**: El modelo genera la siguiente sentencia pero **no verifica la veracidad** de la información (progresiva mejora de este efecto).

¿Qué es Langchain y cuáles son sus componentes?

- **LangChain** es un framework para desarrollar **aplicaciones** impulsadas por modelos de lenguaje grandes (LLMs), como los modelos GPT de OpenAI o PaLM-2 de Google o Gemini LLM.
 - El framework Langchain se consiste en una serie de **módulos** (a veces también denominados componentes).
 - Este curso está diseñado para avanzar lentamente hacia la comprensión de los **agentes** LangChain, que son aplicaciones basadas en LLM extremadamente poderosas que permiten realizar tareas complejas y permiten que un LLM realice acciones (búsqueda en Wikipedia, usar una calculadora, herramientas personalizadas, etc.).
 - Los propios agentes solo requieren unas pocas líneas de código, pero saltar directamente a Agentes puede generar confusión sobre LangChain.
- **Documentación Python Langchain:**
 - <https://python.langchain.com/>

¿Qué es Langchain y cuáles son sus componentes?

MODELO IO

El modelo IO se centra en entradas y salidas básicas de LLM. Esto replica gran parte de la funcionalidad que ya está presente en las **API** predeterminadas, como la API OpenAI.

Ventaja de LangChain: Model IO está **estandarizado**, lo que le permitirá cambiar fácilmente entre LLM en el futuro y admite API de finalización de texto y API de **interfaz de chat**.

CONECTORES DE DATOS

Data Connection se centra en conectar un modelo LLM a una fuente de datos, como sus propios documentos o un almacén de vectores.

Ventaja de LangChain: Intercambie fácilmente almacenes de vectores. Integre con muchas fuentes de datos diferentes para crear objetos de documentos estandarizados a partir de una variedad de fuentes (CSV, PDF, AWS, Google Cloud, etc.).

CADENAS (CHAINS)

Cadenas le permite vincular la salida de un modelo para que sea la entrada de otra llamada de modelo.

Ventaja de LangChain: Encadene fácilmente diferentes llamadas de LLM para separar el trabajo entre modelos, lo que le permitirá intercambiar LLM en medio de una cadena fácilmente.

MEMORIA

La memoria permite que sus modelos retengan el **contexto histórico** de interacciones anteriores.

Ventaja de LangChain: Guarde fácilmente conversaciones históricas o resultados de LLM o modelos de chat y recárguelos para usarlos en el futuro.

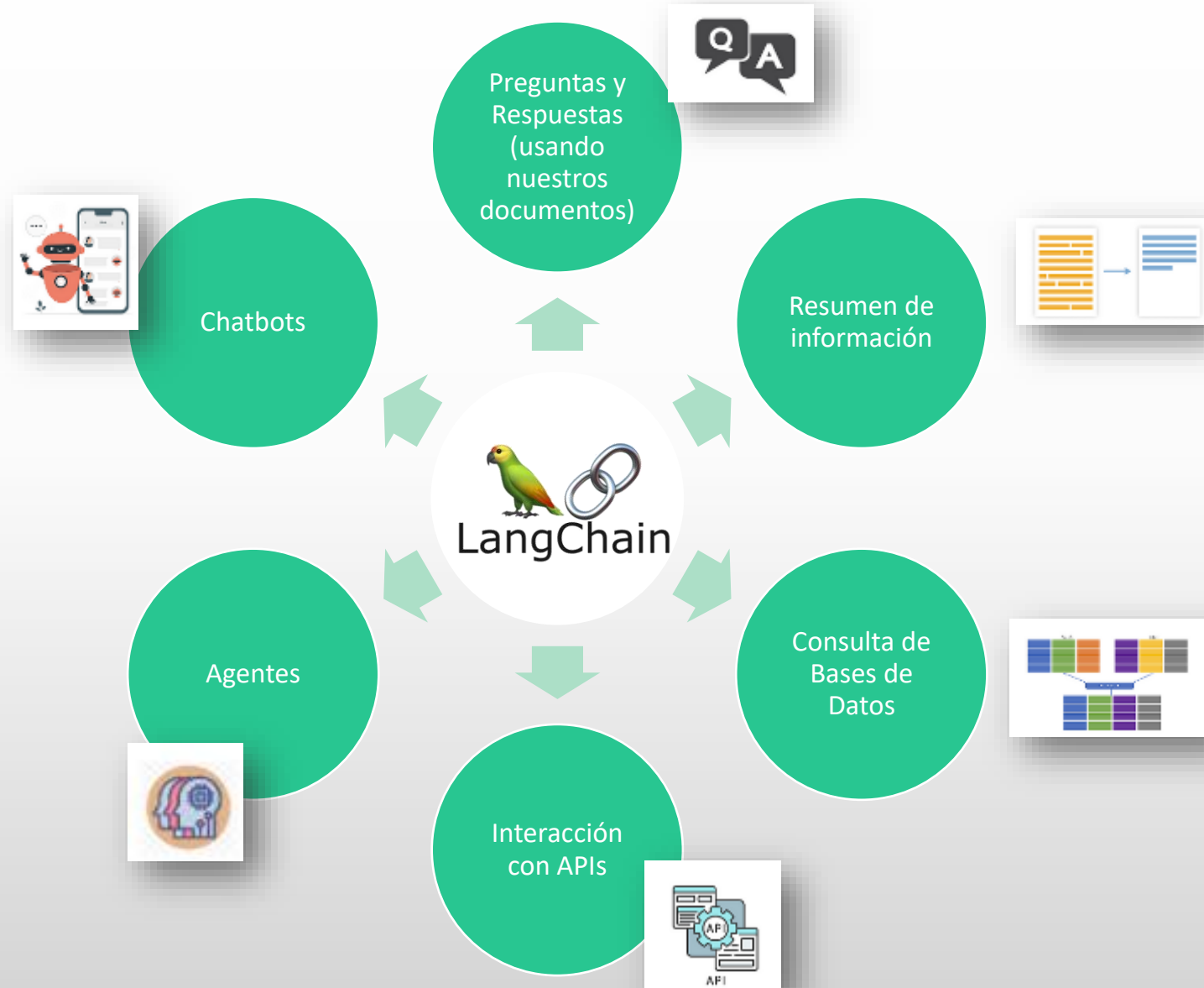
AGENTES

Los agentes son la parte más poderosa de LangChain. Utilizando modelos, conexiones de datos, cadenas y memoria, los agentes pueden utilizar herramientas para razonar solicitudes y **realizar acciones basadas en resultados de observación**.

Ventaja de LangChain: Crea agentes y herramientas personalizadas con solo unas pocas líneas de código

¿Qué es Langchain y cuáles son sus componentes?

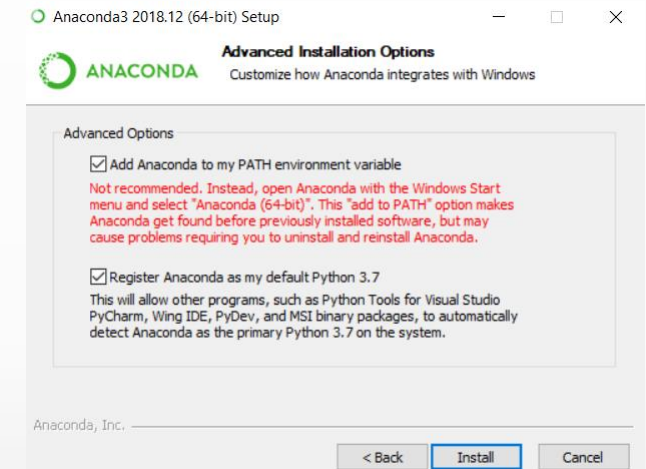
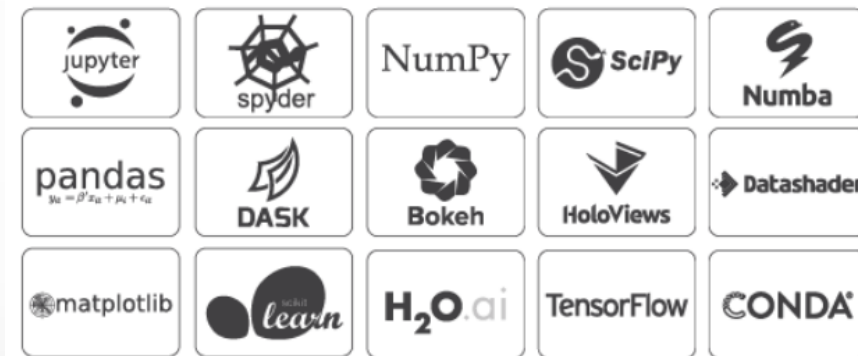
Casos de uso



Instalación de Python y librería Langchain

- **ANACONDA:** Distribución de Python con múltiples librerías pre-instaladas para Data Science.

<https://www.anaconda.com/download>



- **Instalación librería Langchain para Python:**
 - En cmd: pip install langchain
 - En cmd: pip install openai
 - En cmd: pip install langchain_openai

¿Qué es una API?

ChatGPT

resume la historia de Madrid en el siguiente formato:

Inicio:

Construcciones:

Población actual: [en millones de personas]

PIB: [en €]



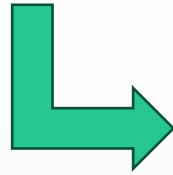
Inicio: Madrid como núcleo urbano continuo comenzó en el siglo IX.

Construcciones: Destaca la fortaleza del siglo IX y la consolidación como capital en 1561.

Población actual: 6.8 millones de personas.

PIB: 241 mil millones de euros.

SOLICITUD



GPT-3.5



RESPUESTA

Nueva aplicación

SOLICITUD

API KEY



GPT-3.5



RESPUESTA

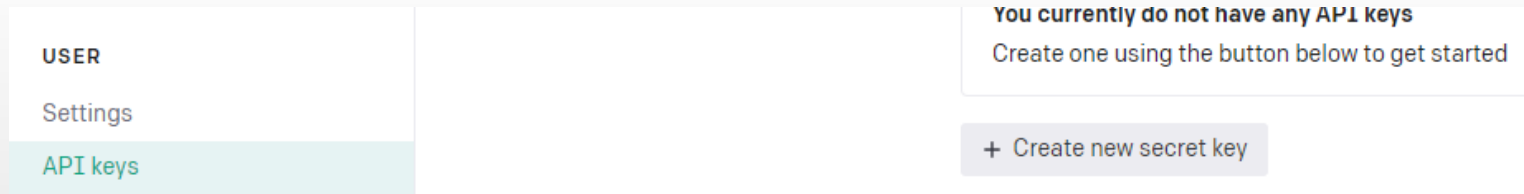
API: Application Program Interface

Conjunto de protocolos y rutinas que permite conectar 2 aplicaciones software

Configuración de la cuenta OpenAI y API Key



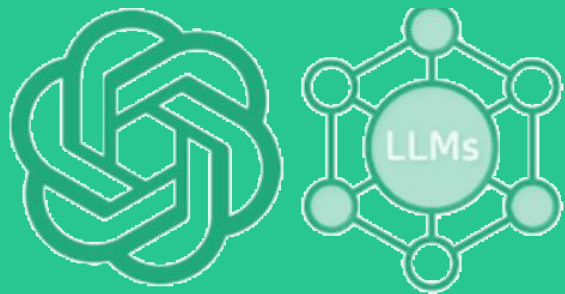
- Ir a <https://platform.openai.com/>
- En “Documentation” se puede ver una extensa documentación de la API incluidos los modelos disponibles.
- En “API reference” tenemos la vía para interactuar mediante sentencias de código con la API (aunque usaremos las funciones de Langchain para hacerlo en lugar de la API nativa)
- Se puede revisar los precios del uso de la API en: <https://openai.com/pricing>
- Para crear una API Key pulsamos en nuestro usuario y “View API Keys” > “Create new secret key”



- Establecer método de pago:



- Indicar límites de uso en “Usage limits”
- **Instalación librería openai en Python con Langchain:**
 - En cmd: pip install openai
 - En cmd: pip install langchain_openai



UNIDAD: MODELOS DE ENTRADA / SALIDA EN LANGCHAIN

Lo que aprenderemos en este bloque....

- ✓ ¿Cómo interactuamos con los LLMs usando Langchain?
- ✓ ¿Cómo crear plantillas de prompts para aumentar la flexibilidad de la entrada al modelo?
- ✓ ¿Cómo parsear la salida del modelo al tipo de datos que necesitamos?
- ✓ ¿Cómo serializamos los resultados y compartimos plantillas de prompts?

Interacción y uso de Langchain con LLMs y Modelos de Chat

- En esta sección, comenzaremos nuestro viaje de aprendizaje de LangChain entendiendo cómo **crear solicitudes de entrada** básicas para modelos y cómo gestionar sus resultados.
- Básicamente, **Langchain necesita poder enviar texto al LLM y también recibir y trabajar con sus resultados**. Esta sección del curso se centra en las funcionalidades básicas y la sintaxis para hacer esto con Langchain.
- El uso de Langchain y el componente **Modelo IO** nos permitirá construir cadenas más adelante, pero también nos dará más **flexibilidad para cambiar de proveedor de LLM en el futuro**, ya que la sintaxis está **estandarizada** en todos los LLM y solo cambian los parámetros o argumentos proporcionados.
- Langchain es compatible con todos los principales LLM (OpenAI, Azure, Anthropic, Google Cloud, etc.)
- Debemos tener en cuenta 2 parámetros importantes en las solicitudes a las APIs de los LLMs:



- Para verificar cómo conectar a los diferentes LLMs integrados en Langchain:
<https://python.langchain.com/v0.2/docs/integrations/chat/>

Plantillas de prompts con Langchain para el modelo de entrada

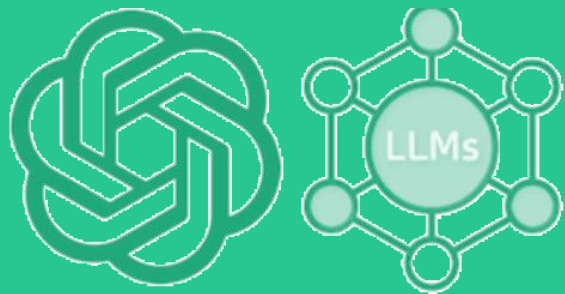
- Las **plantillas** nos permiten configurar y modificar fácilmente nuestras **indicaciones** de entrada para las llamadas de LLM.
- Las plantillas ofrecen un enfoque más sistemático para pasar variables a solicitudes de modelos, en lugar de usar literales de cadena f o llamadas `.format()`, **PromptTemplate las convierte en nombres de parámetros de función que podemos pasar.**
- Es recomendable usar plantillas para estandarizar los mensajes que enviamos a los LLMs para mayor flexibilidad y facilidad en futuros usos.

Parsear y procesar la salida

- A menudo, al conectar la salida de un LLM (modelo de lenguaje grande), necesitas que esté en un formato particular, por ejemplo, puedes querer un objeto datetime de Python o un objeto JSON.
- LangChain viene con utilidades de análisis que te permiten convertir fácilmente las salidas en tipos de datos precisos gracias a los **parseadores**.
- **Elementos clave:**
 - Método `parse()`: Método concreto para evaluar la cadena de texto (string) y parsearla al tipo deseado.
 - `Format_instructions`: Una cadena de texto extra que Langchain añade al final del prompt para asistir y facilitar la interpretación por el LLM del formato deseado.
- Si no consigues el resultado parseado correctamente (por ejemplo, la respuesta del LLM es más extensa que únicamente una fecha que quieras parsear), hay 2 soluciones:
 - Usar parseador “Auto-fix”.
 - Usar un “system prompt” para dar mayor detalle al LLM de cómo debe actuar y responder globalmente.

Serialización de prompts (guardar y cargar)

- Es posible que desee **guardar, compartir o cargar objetos de prompts**.
- **Langchain** permite guardar fácilmente plantillas de mensajes como archivos **JSON** para leer o compartir, muy útil cuando hay plantillas complejas que distribuir o reutilizar.



UNIDAD : CONECTORES DE DATOS EN LANGCHAIN

Lo que aprenderemos en este bloque....

- ✓ ¿Cómo cargar fuentes de datos de todo tipo con Langchain?
- ✓ ¿Cómo transformar documentos y fragmentarlos?
- ✓ ¿Cómo convertir documentos en vectores a partir de incrustaciones (embeddings)?
- ✓ ¿Cómo almacenar los datos (internos y externos) en una base de datos vectorizada?
- ✓ ¿Cómo realizar consultas a la base de datos vectorizada y mejorar los resultados con LLMs?

Cargadores de documentos

- Langchain viene con herramientas de carga integradas para cargar rápidamente archivos en su propio objeto *Documento*.
- Muchos de estos cargadores requieren otras bibliotecas, por ejemplo, la carga de PDF requiere la biblioteca *pypdf* y la carga de HTML requiere la biblioteca *Beautiful Soup*. Asegurar de instalar las bibliotecas requeridas antes de usar el cargador (los cargadores informarán si no pueden encontrar las bibliotecas instaladas).
- **Instalación librería Langchain community para loaders en Python:**
 - En cmd: `pip install langchain-community`

Documentación document loaders:

https://python.langchain.com/v0.2/docs/integrations/document_loaders/

Cargadores de documentos – integraciones con otras plataformas

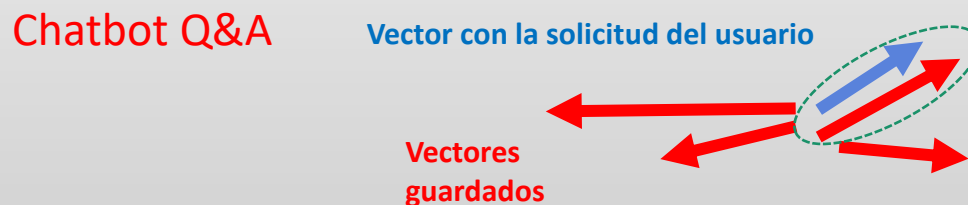
- Los cargadores de documentos etiquetados como "integraciones" pueden considerarse esencialmente lo mismo que los cargadores de documentos normales, pero están integrados con otras plataformas como:
 - Plataforma de terceros (como Google Cloud, AWS, Google Drive, Dropbox,...)
 - Base de datos (como MongoDB)
 - Sitio web específico, como **Wikipedia**.
 - Permiten cargar vídeos de Youtube (por ejemplo, crear una aplicación de preguntas y respuestas en base a vídeos de Youtube), conversaciones de WhatsApp y un sinfín de posibilidades.

Documentación document loaders - integraciones:

https://python.langchain.com/v0.2/docs/integrations/document_loaders/

Transformación de documentos

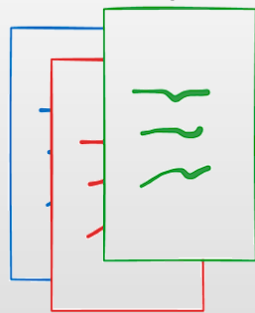
- Hay que tener en cuenta que después de cargar un objeto Documento desde una fuente, terminará con cadenas de texto desde el campo *page_content*.
- En determinadas situaciones, la longitud de las cadenas puede ser demasiado grande para alimentar un modelo (por ejemplo, límite de 8k tokens ~6k palabras).
- Langchain proporciona **transformadores de documentos** que permiten dividir fácilmente cadenas del *page_content* en **fragmentos** (*chunks*).
- Estos fragmentos servirán más adelante además como componentes útiles en forma de **vectores a partir de una incrustación** (*embeddings*), que luego podremos buscar utilizando una similitud de distancia más adelante. Por ejemplo, si queremos alimentar un LLM con contexto adicional para que sirva como chatbot de preguntas y respuestas, si tenemos varios vectores guardados cada uno con una información diferente, la búsqueda será más rápida puesto que se hará un cálculo del vector guardado que tiene mayor similaridad en lugar de buscar en todos los datos globales.



Incrustación de texto y creación de vectores (embeddings)

- Langchain admite muchas incrustaciones de texto, que pueden convertir directamente texto en una representación **vectorizada** incrustada.
- Más adelante podemos almacenar estos vectores incrustados y realizar búsquedas de similitud entre nuevos documentos vectorizados o cadenas **contra el almacén de vectores**.
- En resumen, **los modelos incrustados crean una representación vectorial de un fragmento de texto**. Puedes pensar en un vector como una matriz de números que captura el significado semántico del texto. Al representar el texto de esta manera, puede realizar operaciones matemáticas que le permiten hacer cosas como buscar otras partes del texto que tengan un significado más similar.

Fragmentos (chunks)



Vectores

$[x_i, y_i, z]$
 $[x_i, y_i, z]$
 $[x_i, y_i, z]$

<https://python.langchain.com/v0.2/docs/concepts/#embedding-models>

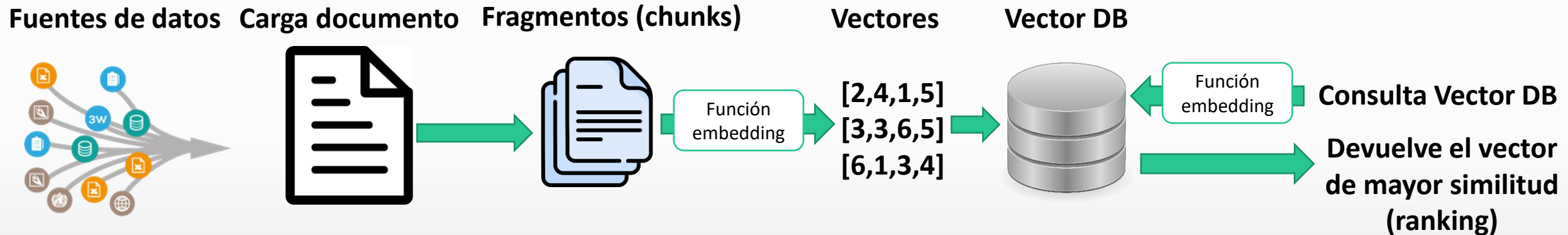
- **NOTA:** Los diferentes modelos de incrustación puede que no interactúen entre sí, lo que significa que necesitaría volver a incrustar un conjunto completo de documentos si cambiara de modelo de incrustación en el futuro (usaremos el de OpenAI por ser el más popular).

Almacenamiento de vectores en base de datos

- Hasta ahora hemos creado incrustaciones (embeddings) en memoria RAM como una lista de Python, ¿cómo podemos asegurarnos de que estas incorporaciones persistan en alguna solución de almacenamiento más permanente?
- Para ello, utilizamos un almacén de vectores, también conocido como **base de datos de vectores**, sus aspectos claves:
 - Puede almacenar grandes vectores de N dimensiones.
 - Puede indexar directamente un vector incrustado y **asociarlo** a su documento string.
 - Se puede "consultar", lo que permite una **búsqueda** de similitud de coseno entre un nuevo vector que no está en la base de datos y los vectores almacenados.
 - Puede agregar, actualizar o eliminar fácilmente nuevos vectores.
 - Al igual que con los LLM y los modelos de chat, **Langchain** ofrece muchas opciones diferentes para almacenes de vectores.
 - Usaremos una base de datos de vectores open-source SKLearn, pero gracias a Langchain, la sintaxis es estándar para el resto de BD.

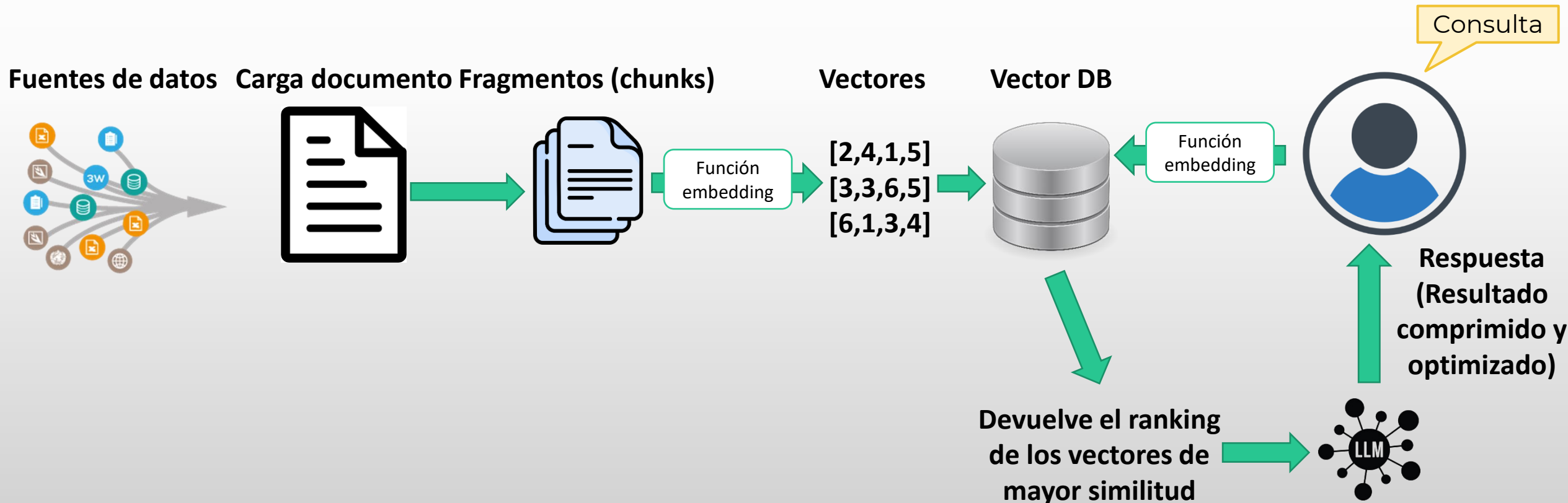
<https://python.langchain.com/v0.2/docs/integrations/vectorstores/>

Almacenamiento de vectores en base de datos



Compresión y optimización de resultados a partir de LLMs

- Anteriormente devolvimos la **totalidad del documento vectorizado**. Lo ideal sería pasar este documento como contexto a un LLM para obtener una respuesta más relevante y optimizada (comprimida).
- No estamos realizando compresión en el sentido tradicional, sino que utilizamos un LLM para tomar una salida de texto de un documento de mayor tamaño y la **limpia / optimiza en una salida más corta y relevante**.





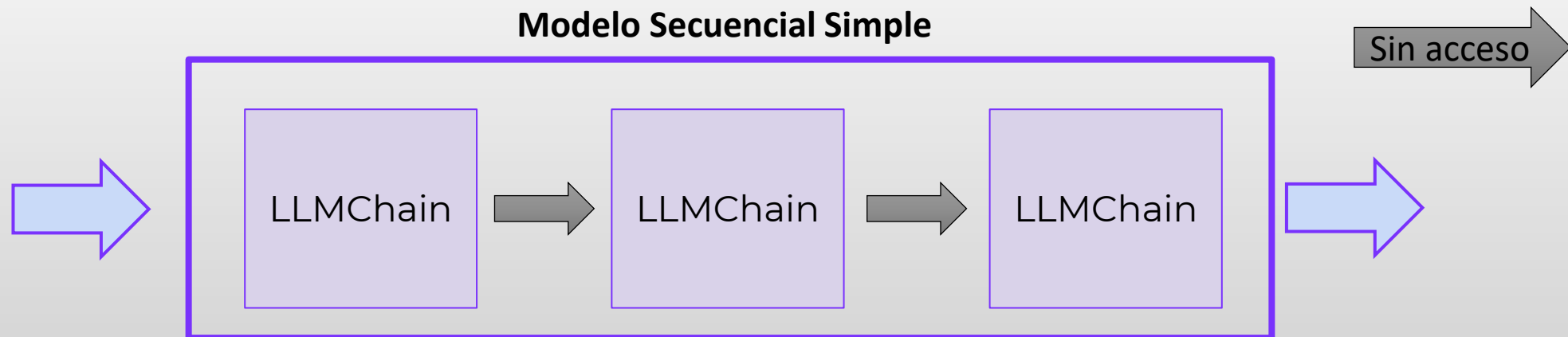
UNIDAD : CADENAS EN LANGCHAIN

Lo que aprenderemos en este bloque....

- ✓ ¿Cómo crear cadenas para construir aplicaciones complejas?
- ✓ ¿Cómo crear un modelo secuencial para combinar cadenas?
- ✓ ¿Cómo enrutar a la mejor cadena?
- ✓ ¿Cómo crear cadenas avanzadas de transformación y preguntas/respuestas sobre nuestros datos vectorizados?

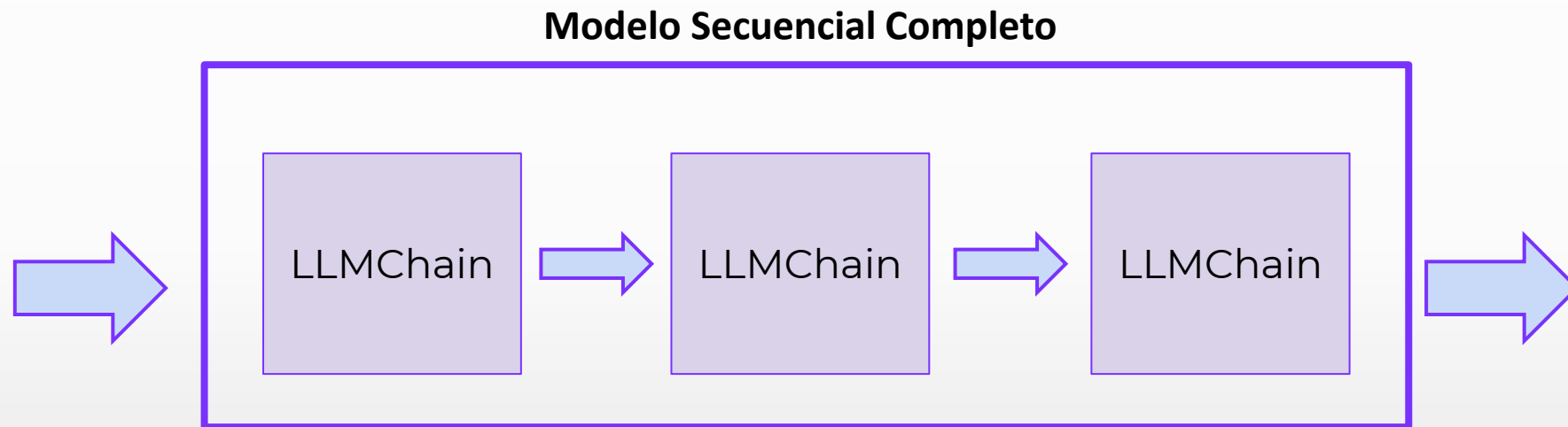
¿Qué son las cadenas y cómo crear el primero modelo de cadena secuencial simple?

- Las **cadenas** nos permiten combinar múltiples componentes para crear una aplicación única y coherente. Por ejemplo, podemos crear una cadena que tome la **entrada del usuario**, la formatee con PromptTemplate y luego pase la respuesta formateada a un **LLM**. Podemos construir cadenas más complejas combinando varias cadenas o combinando cadenas con otros componentes, por tanto, las **cadenas** nos permite vincular la salida de una llamada LLM con la entrada de otra llamada LLM.
- Las cadenas tienen un componente básico conocido como objeto **LLMChain**. Podemos pensar en LLMChain como una simple llamada a LLM que tendrá una entrada (normalmente el prompt o solicitud) y una salida (el resultado).
- Si encadenamos varios LLMChain podemos tener un modelo secuencial, el modelo secuencial simple tiene solo 1 entrada global y 1 salida global, no nos proporciona los resultados intermedios. Cada objeto LLMChain intermedio solo tiene 1 entrada y 1 salida, por tanto la salida de 1 LLMChain será la entrada del siguiente:

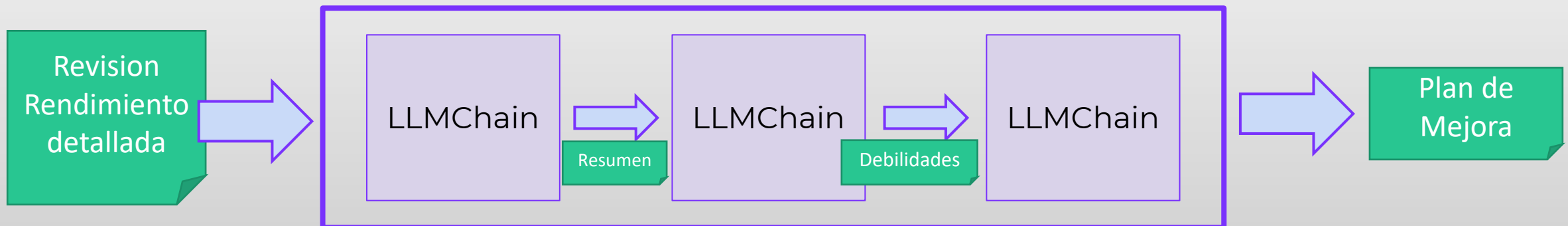


Construcción del Modelo Secuencial Completo

- Las cadenas secuenciales completas son muy similares a las simples vistas anteriormente, pero nos permiten tener **acceso a todas las salidas de las LLMChains internas**.

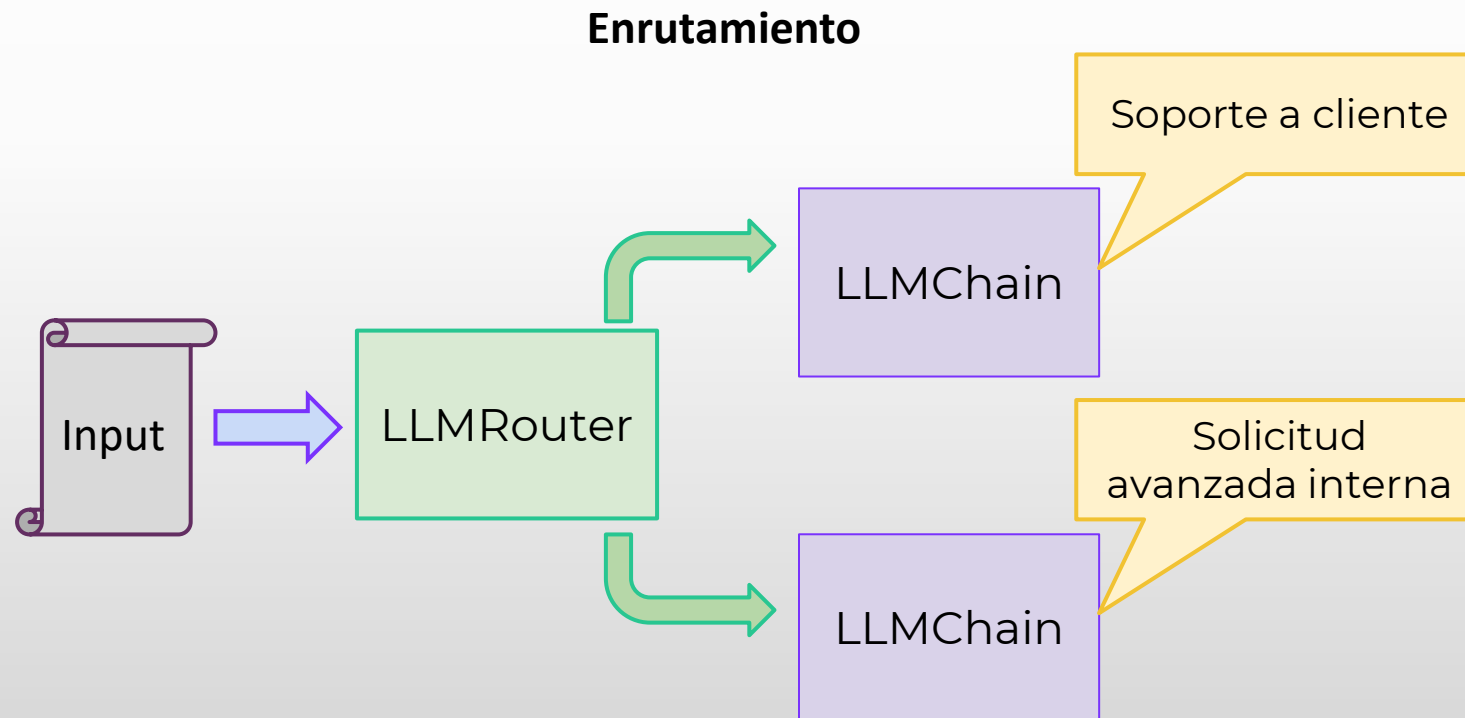


Caso de uso: Rendimiento empleados



Enrutamiento a cadenas con LLMRouterChain

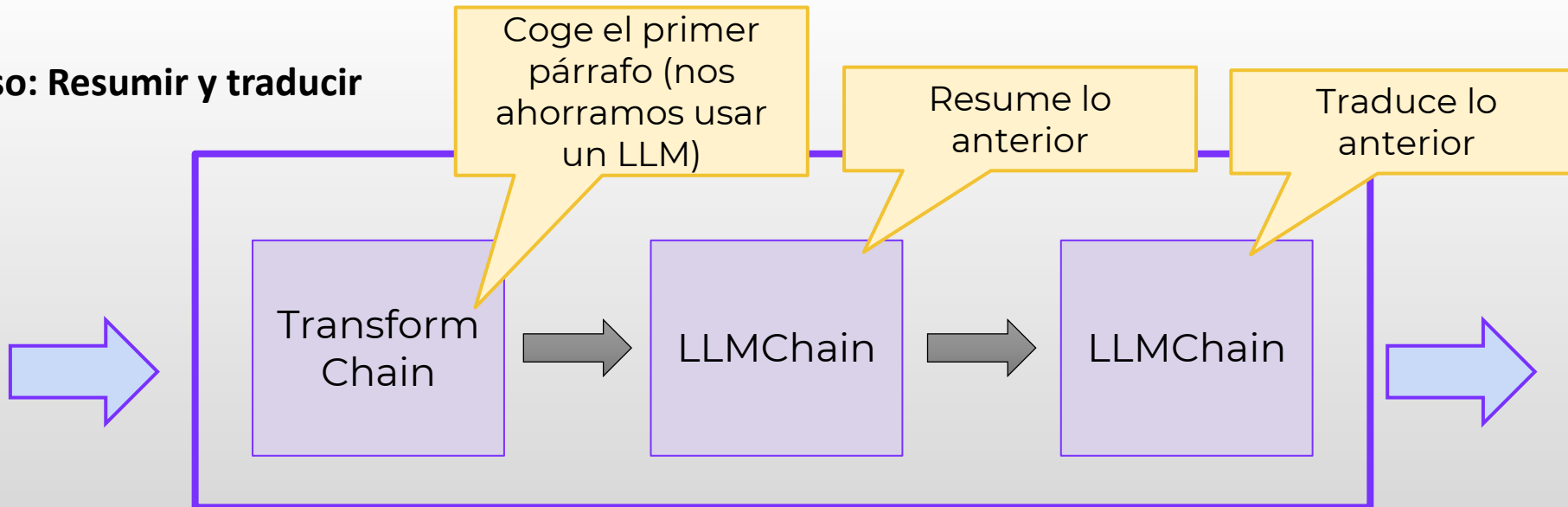
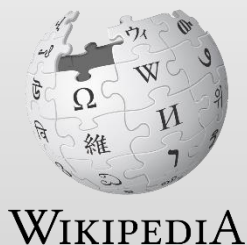
- LLMRouterChains puede recibir una entrada y redirigirla a la secuencia LLMChain más **apropiada**.
- El enrutador acepta múltiples LLMChains de destino potencial y luego, a través de un mensaje especializado, el enrutador leerá la entrada inicial y generará un diccionario específico que coincida con una de las posibles cadenas de destino para continuar con el procesamiento.



Cadenas de Transformación

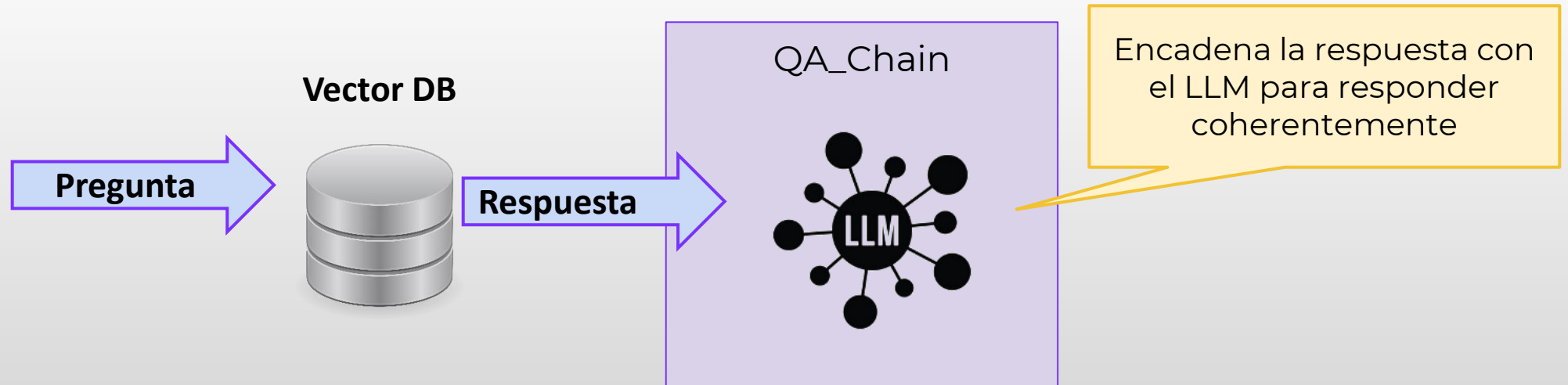
- Las "TransformChain" se refieren a un tipo específico de cadena de transformación que se utiliza para **modificar, manipular o transformar datos** en el proceso de construcción de aplicaciones basadas en modelos de lenguaje.
- Ejemplos:**
 - Normalización de texto: Limpiar y estandarizar el texto de entrada para asegurar consistencia.
 - Formateo: Cambiar el formato del texto para adaptarlo a necesidades específicas.
 - Filtrado: Eliminar información innecesaria o irrelevante.

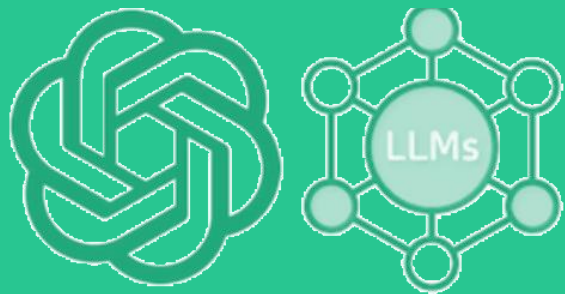
Caso de uso: Resumir y traducir



Cadenas para Preguntas y Respuestas sobre nuestros datos

- La cadena “load_qa_chain” se utiliza para cargar y configurar una cadena de **preguntas y respuestas (Q&A)**. Esta cadena puede combinar múltiples componentes de procesamiento de lenguaje natural para responder preguntas basadas en un contexto específico sin tener que hacerlo manualmente.
- La clave es que proporciona una respuesta directa y coherente de la pregunta realizada a partir de nuestra base de datos de vectores en lugar de únicamente devolver el vector de mayor similitud.
- Muy común usarla cuando estamos creando un **Retrieval-Augmented Generation - RAG** (Generación Aumentada por Recuperación) para obtener información y respuestas basadas en nuestra propia base de datos de vectores.





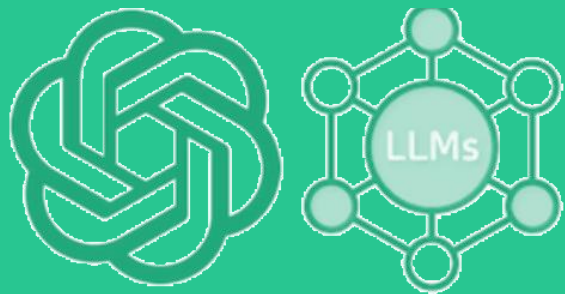
UNIDAD : MEMORIA EN LANGCHAIN

Lo que aprenderemos en este bloque....

- ✓ ¿Cómo almacenar las conversaciones humano - AI?
- ✓ ¿Cómo guardar en un objeto de memoria las conversaciones con el LLM?
- ✓ ¿Cómo limitar o resumir lo que guardamos en el objeto de memoria?

¿Qué es la memoria en Langchain y qué tipos de memoria podemos implementar?

- Cuando LangChain se refiere al término "**memoria**", generalmente se refiere a realizar un seguimiento del historial de interacción de mensajes.
- Existen varios tipos:
 - **ChatMessageHistory**: Guardar el histórico de mensajes de un chat con los métodos `add_user_message` y `add_ai_message`, debemos guardarlo manualmente tras cada interacción.
 - **ConversationBufferMemory**: A partir de una cadena de tipo `ConversationalChain`, guardamos en un objeto de memoria todos los mensajes de la conversación.
 - **ConversationBufferWindowMemory**: Igual que el anterior pero podemos especificar una ventana de `k` interacciones a guardar en lugar de todos los mensajes.
 - **ConversationSummaryMemory**: En lugar de guardar los mensajes literalmente, se realiza un resumen de la memoria en base a todo el historia, con ello reducimos drásticamente el tamaño de la memoria para conversaciones muy largas.



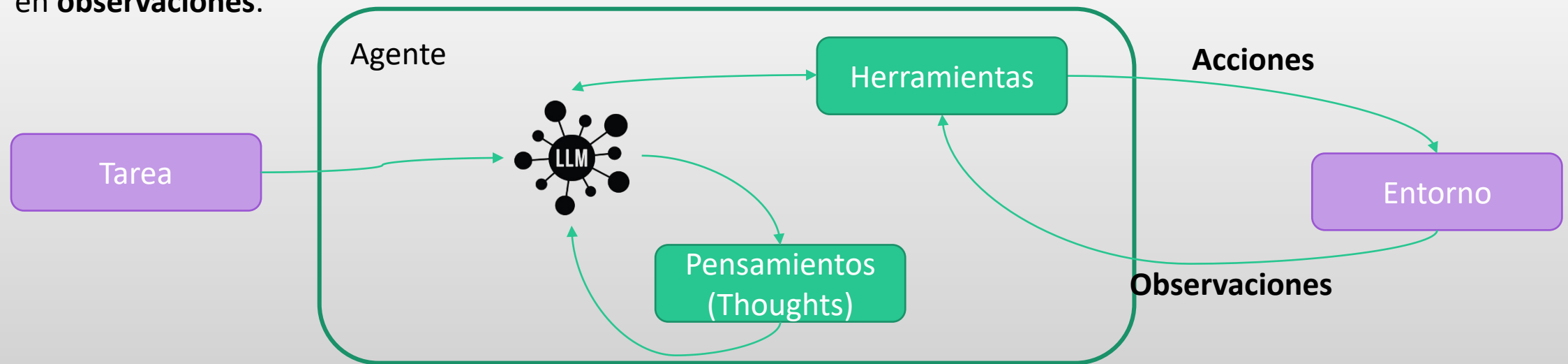
UNIDAD : AGENTES EN LANGCHAIN

Lo que aprenderemos en este bloque....

- ✓ ¿Qué son los agentes y su funcionamiento?
- ✓ ¿Cómo crear agentes asistidos con motores de búsqueda?
- ✓ ¿Cómo crear agentes programadores de código y conversacionales?
- ✓ ¿Cómo usar herramientas personalizadas por los agentes?
- ✓ ¿Cómo crear potentes agentes reales?

¿Qué son los agentes y cómo implementar un primer caso de uso?

- Los agentes son una de las partes más novedosas de LangChain, pero ofrecen un enorme potencial para **aplicaciones basadas en LLM**.
- Al combinar lo que ya aprendimos sobre Model IO, conexiones de datos y cadenas, ya hemos abordado aplicaciones similares a agentes, pero los agentes facilitan la creación de estas aplicaciones siendo además más robustas.
- Básicamente, los agentes permiten a los LLM conectarse a **herramientas** (por ejemplo, Wikipedia, Calculadora, Búsqueda de Google, etc.) y llevar a cabo un enfoque estructurado para completar una tarea basada en **ReAct** (razonamiento y actuación).
- Las indicaciones de ReAct pueden ayudar a un LLM a **razonar** sobre una tarea y realizar acciones correctas basadas en **observaciones**:



¿Qué son los agentes y cómo implementar un primer caso de uso?

- Al Agente se le asigna una **tarea** y puede razonar qué **herramientas** son apropiadas para usar y luego puede utilizar esos resultados para continuar a través de una cadena interna hasta que resuelva la tarea.

- Ejemplo:

```
agent.run("Dime cuánto es 1598 multiplicado por 1983 y después sumas 1000")

> Entering new AgentExecutor chain...
I should use the Calculator tool to multiply 1598 by 1983 and then add 1000.
Action: Calculator
Action Input: multiply(1598, 1983)
Observation: Answer: 3168834
Thought: Now I need to add 1000 to the result.
Action: Calculator
Action Input: 3168834 + 1000
Observation: Answer: 3169834
Thought: I now know the final answer
Final Answer: 3169834

> Finished chain.

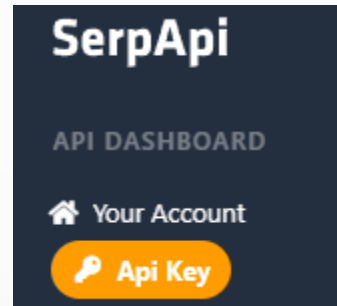
'3169834'
```

- Los agentes pueden ser extremadamente poderosos, especialmente si los combinamos con **nuestras propias herramientas personalizadas**.
- Imagina un Agente con acceso a documentos corporativos internos y la capacidad de realizar búsquedas relevantes externas, de repente, tendrás un asistente corporativo muy poderoso con información interna y externa para responder preguntas (de clientes, de personal interno,...).

https://python.langchain.com/v0.1/docs/modules/agents/agent_types/

Creación de agente potenciado con motores de búsqueda

- Registrarse en: <https://serpapi.com/>
- Después accedemos a la API Key (de momento es gratuito):



- Instalar librería: `pip install google-search-results`
- Tendremos acceso a motores de búsqueda que serán más potentes que “solo” disponer de la información del LLM.

Documentación herramientas:

<https://python.langchain.com/v0.1/docs/integrations/tools/>

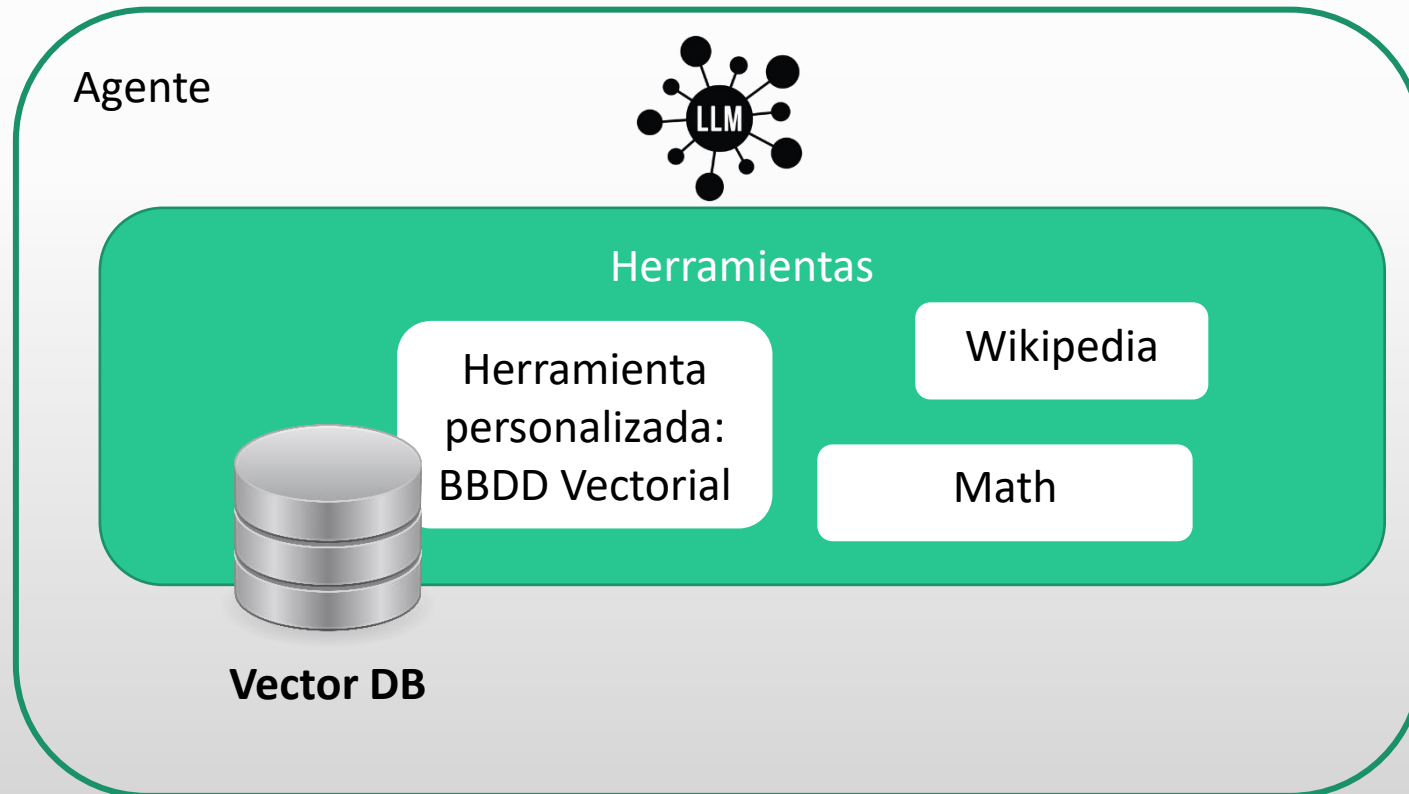
Creación de herramientas personalizadas

- Podemos definir nuestras propias herramientas (tools) para ser usadas por el agente.
- Es muy importante definir bien el ***docstring*** (descripción de la función) puesto que en base a ello el agente seleccionará o no esa herramienta.
- El uso de herramientas personalizadas expande el uso de los agentes, podríamos incluso definir herramientas que conecten con APIs internas de nuestra empresa para determinadas tareas.

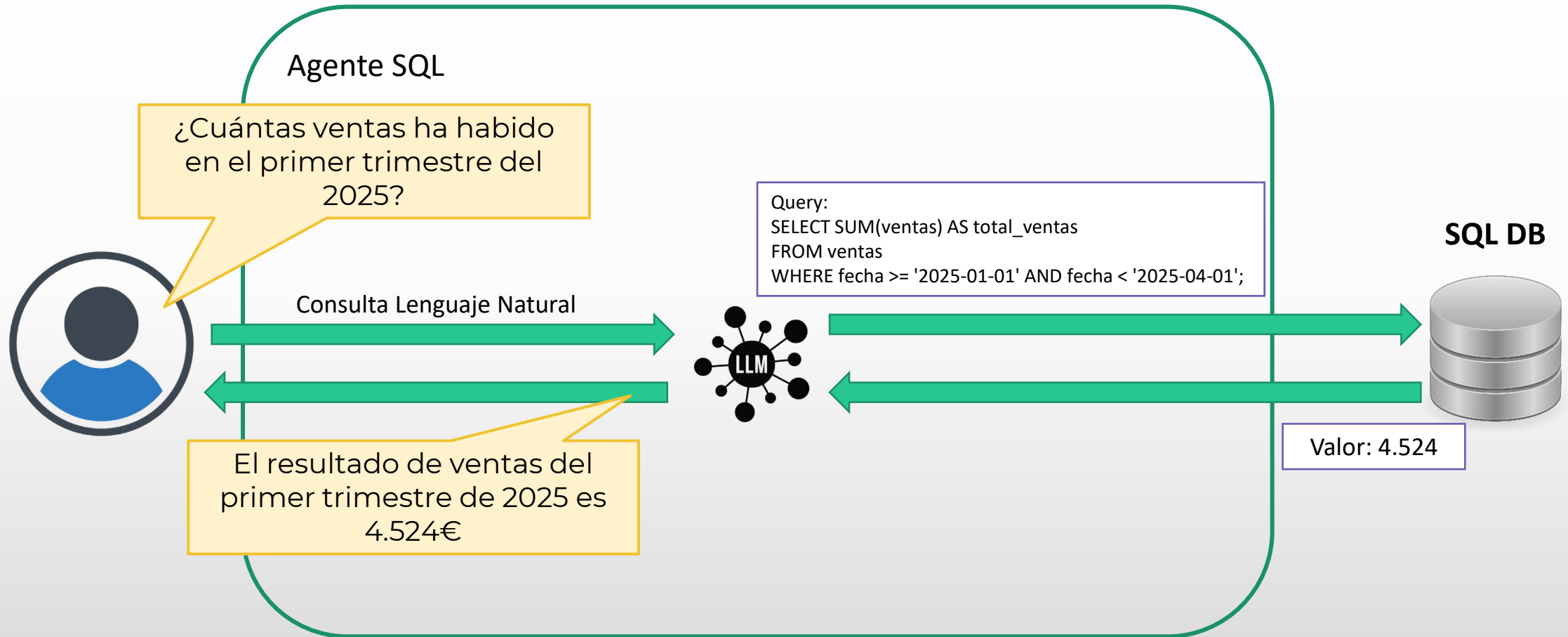


PROYECTO: Creación de Agente Chatbot con memoria a partir de sistema RAG con nuestra BD Vectorial

- Vamos a crear un agente que combine el potencial de las BD Vectorizada con nuestros propios documentos y el resto de herramientas.
- El agente debe verificar si la herramienta apropiada es la personalizada que creamos que obtendrá datos de la BBDD Vectorial o, sin embargo, debe usar otras herramientas como Wikipedia para consultar información o bien el propio conocimiento del LLM.



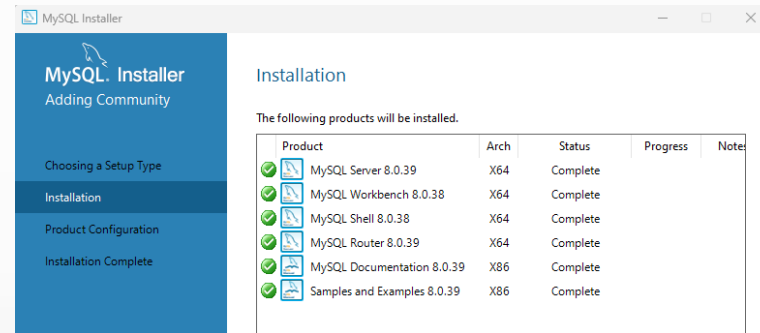
PROYECTO: Creación de Agente para Análisis automático SQL a partir de consultas en lenguaje natural



PROYECTO: Creación de Agente para Análisis automático SQL a partir de consultas en lenguaje natural

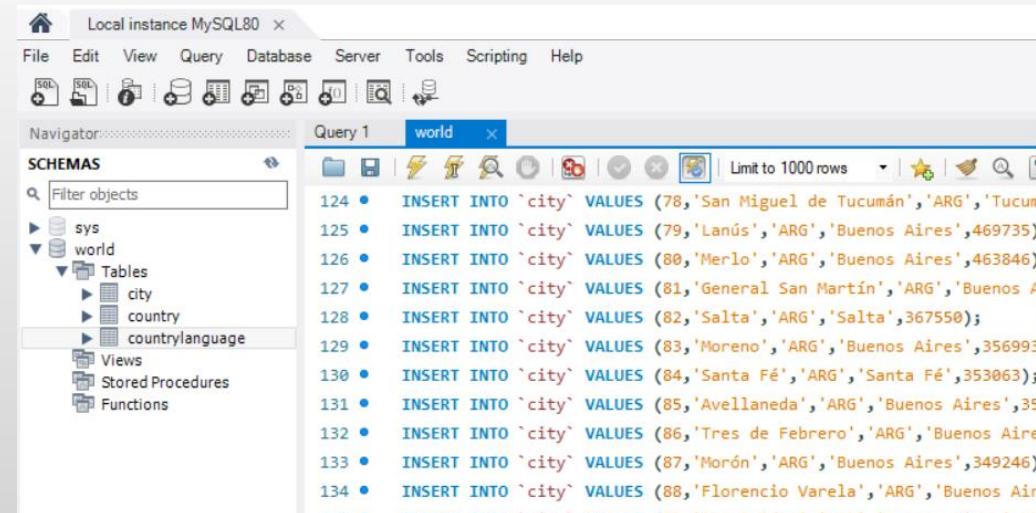
Opcional: Instalación MySQL BD

Descargar MySQL Community Server (BBDD) / MySQL Workbench (Diseño de BBDD visual): Elegir instalación “Full”: <https://dev.mysql.com/downloads/>



Crear BBDD MySQL

File > Open SQL Script > Execute



Gracias

- Iván Pinar Domínguez
- www.linkedin.com/in/ivanpinar

